

编 号	题 号	标 题	知 识 点	代 码 作 者
例 7-23	UVa 11419	SAM I AM	二分图最大匹配；最小点覆盖	刘汝佳
例 7-24	POJ 2771	Guardian of Decency	二分图最大独立集	刘汝佳
例 7-25	POJ 2060	Taxi Cab Scheme	DAG 最小路径覆盖	刘汝佳
例 7-26	UVa 1349	Optimal Bus Route Design	后继模型；二分图最小权匹配	刘汝佳
例 7-27	UVa 11383	Golden Tiger Claw	KM 算法；可行顶标	刘汝佳
例 7-28	POJ 3565	Ants	二分图最佳匹配	刘汝佳
例 7-29	UVa 1006	Fixed Partition Memory Management	图论建模；二分图最佳匹配	刘汝佳

7.5 网络流问题

本节通过一些经典题目介绍网络流相关模型的代码实现。本节给出的代码在大多数比赛中已经完全够用。

例7-30 【最大流；ISAP】网络扩容（Frequency Hopping, UVa 11248）

给定一个有向网络，包含 $ECE \leq 10\,000$ 条边，每条边 (u, v) 均有一个容量 cap ($1 \leq u, v \leq N$, $1 \leq \text{cap} \leq 5000$)。问是否存在一个从点 1 到点 N 、流量为 C ($1 \leq N \leq 100$, $C \leq 2 \times 10^9$) 的流。如果不存在，是否可以恰好修改一条弧的容量，使得存在这样的流。

如果流已经存在，输出 “possible”；如果目前不存在流，但可以通过修改恰好一条弧的容量得到，则输出 “possible option:” 和这些弧的列表（按照起点从小到大排序，起点相同时按照终点从小到大排序）；否则，输出 “not possible”。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <queue>
#include <vector>
using namespace std;

const int NN = 100 + 10, INF = 1e9;
struct Edge {
    int from, to, cap, flow;
};
bool operator<(const Edge& a, const Edge& b) {
    return a.from < b.from || (a.from == b.from && a.to < b.to);
}
struct ISAP {
    int n, m, s, t;
    vector<Edge> edges;
    vector<int> G[NN]; // 邻接表, G[i][j]表示结点 i 的第 j 条边在 e 数组中的序号
    bool vis[NN];      // BFS 使用
    int d[NN];         // 从起点到 i 的距离
    int cur[NN];       // 当前弧指针
    int p[NN];         // 可增广路上的上一条弧
    int num[NN];       // 距离标号计数

    void AddEdge(int from, int to, int cap) {
        edges.push_back((Edge) {from, to, cap, 0});
        edges.push_back((Edge) {to, from, 0, 0});
        m = edges.size();
        G[from].push_back(m - 2), G[to].push_back(m - 1);
    }
};

```

```

}

bool BFS() {
    fill_n(vis, n + 1, false);
    queue<int> Q;
    Q.push(t), vis[t] = 1, d[t] = 0;
    while (!Q.empty()) {
        int x = Q.front();
        Q.pop();
        for (size_t i = 0; i < G[x].size(); i++) {
            Edge& e = edges[G[x][i] ^ 1];
            if (!vis[e.from] && e.cap > e.flow)
                vis[e.from] = 1, d[e.from] = d[x] + 1, Q.push(e.from);
        }
    }
    return vis[s];
}

void ClearAll(int n) {
    this->n = n;
    for (int i = 0; i < n; i++) G[i].clear();
    edges.clear();
}

void ClearFlow() {
    for (size_t i = 0; i < edges.size(); i++) edges[i].flow = 0;
}

int Augment() {
    int x = t, a = INF;
    while (x != s) {
        Edge& e = edges[p[x]];
        a = min(a, e.cap - e.flow), x = edges[p[x]].from;
    }
    x = t;
    while (x != s)
        edges[p[x]].flow += a, edges[p[x] ^ 1].flow -= a, x = edges[p[x]].from;
    return a;
}

int Maxflow(int s, int t, int need) {
    this->s = s, this->t = t;
    int flow = 0;
    BFS();
    fill_n(num, n + 1, 0);
    for (int i = 0; i < n; i++) num[d[i]]++;
    int x = s;
    fill_n(cur, n + 1, 0);
    while (d[s] < n) {

```

```

    if (x == t) {
        flow += Augment();
        if (flow >= need) return flow;
        x = s;
    }
    int ok = 0;
    for (size_t i = cur[x]; i < G[x].size(); i++) {
        Edge& e = edges[G[x][i]];
        if (e.cap > e.flow && d[x] == d[e.to] + 1) { // Advance
            ok = 1, p[e.to] = G[x][i], cur[x] = i; // 注意
            x = e.to;
            break;
        }
    }
    if (!ok) { // Retreat
        int m = n - 1; // 初值注意
        for (size_t i = 0; i < G[x].size(); i++) {
            Edge& e = edges[G[x][i]];
            if (e.cap > e.flow) m = min(m, d[e.to]);
        }
        if (--num[d[x]] == 0) break;
        num[d[x] = m + 1]++, cur[x] = 0; // 注意
        if (x != s) x = edges[p[x]].from;
    }
}
return flow;
}

vector<int> Mincut() { // 最大流后调用
    BFS();
    vector<int> ans;
    for (size_t i = 0; i < edges.size(); i++) {
        Edge& e = edges[i];
        if (!vis[e.from] && vis[e.to] && e.cap > 0) ans.push_back(i);
    }
    return ans;
}

void Reduce() {
    for (size_t i = 0; i < edges.size(); i++) edges[i].cap -= edges[i].flow;
}

void print() {
    printf("Graph:\n");
    for (size_t i = 0; i < edges.size(); i++) {
        const Edge& e = edges[i];
        printf("%d->%d, %d, %d\n", e.from, e.to, e.cap, e.flow);
    }
}

```

```

};

ISAP g;
void solve(int n, int c) {
    int flow = g.Maxflow(0, n - 1, INF);
    if (flow >= c) {
        puts("possible");
        return;
    }
    vector<int> cut = g.Mincut();
    g.Reduce();
    vector<Edge> ans;
    for (size_t i = 0; i < cut.size(); i++) {
        Edge& e = g.edges[cut[i]];
        e.cap = c, g.ClearFlow();
        if (flow + g.Maxflow(0, n - 1, c - flow) >= c) ans.push_back(e);
        e.cap = 0;
    }
    if (ans.empty()) {
        puts("not possible");
        return;
    }
    sort(ans.begin(), ans.end());
    printf("possible option: (%d,%d)", ans[0].from + 1, ans[0].to + 1);
    for (size_t i = 1; i < ans.size(); i++)
        printf(", (%d,%d)", ans[i].from + 1, ans[i].to + 1);
    puts("");
}

int main() {
    for (int n, e, c, kase = 1; scanf("%d%d%d", &n, &e, &c) == 3 && n; kase++) {
        g.ClearAll(n);
        for (int i = 0, b1, b2, fp; i < e; i++)
            scanf("%d%d%d", &b1, &b2, &fp), g.AddEdge(b1 - 1, b2 - 1, fp);
        printf("Case %d: ", kase);
        solve(n, c);
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.6.4 节例题 30
同类题目：Island Transport, HDU 4280
*/

```

例7-31 【最大流；Dinic；图论建模】运送超级计算机（Bring Them There, NEERC 2003,POJ 1895）

宇宙中有 N 个星球，你的任务是用最短的时间把 k 个超级计算机从星球 S 运送到星球 T ，其中 $2 \leq N \leq 50$ ， $1 \leq k \leq 50$ ， $1 \leq S, T \leq N$ ， $S \neq T$ 。每个超级计算机需要一整艘飞船来运输。星球之间有 M （ $1 \leq M \leq 200$ ）条双向隧道，每条隧道需要一天时间来通过，且不能有两艘飞船同时使用同一条隧道。隧道不会连接两个相同的星球，且每一对星球之间最多只有一条隧道。

输出第一行为最少需要的天数 L 。以下 L 行，每行描述一天的运输行动。每行第一个整数为 C_i ，即当天移动的飞船数。接下来有 C_i 对整数 (A, B) ，表示有一艘飞船从星球 A 出发到达星球 B 。输入保证有解。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <queue>
#include <vector>
using namespace std;

const int maxn = 5000 + 10;
const int INF = 1e9;
struct Edge {
    int from, to, cap, flow;
};
bool operator<(const Edge& a, const Edge& b) {
    return a.from < b.from || (a.from == b.from && a.to < b.to);
}

struct Dinic {
    int n, m, s, t;
    vector<Edge> edges; // 边数的两倍
    vector<int> G[maxn]; // 邻接表, G[i][j]表示结点 i 的第 j 条边在 e 数组中的序号
    bool vis[maxn]; // BFS 使用
    int d[maxn]; // 从起点到 i 的距离
    int cur[maxn]; // 当前弧指针

    void init() { edges.clear(); }

    void clearNodes(int a, int b) {
        for (int i = a; i <= b; i++) G[i].clear();
    }

    void AddEdge(int from, int to, int cap) {
        edges.push_back((Edge) {from, to, cap, 0});
        edges.push_back((Edge) {to, from, 0, 0});
        m = edges.size();
        G[from].push_back(m - 2), G[to].push_back(m - 1);
    }

    bool BFS() {
        memset(vis, 0, sizeof(vis));

```

```

queue<int> Q;
Q.push(s), vis[s] = true, d[s] = 0;
while (!Q.empty()) {
    int x = Q.front();
    Q.pop();
    for (size_t i = 0; i < G[x].size(); i++) {
        Edge& e = edges[G[x][i]];
        if (!vis[e.to] && e.cap > e.flow)
            vis[e.to] = true, d[e.to] = d[x] + 1, Q.push(e.to);
    }
}
return vis[t];
}

int DFS(int x, int a) {
    if (x == t || a == 0) return a;
    int flow = 0, f;
    for (int& i = cur[x]; i < G[x].size(); i++) {
        Edge& e = edges[G[x][i]];
        if (d[x] + 1 == d[e.to] && (f = DFS(e.to, min(a, e.cap - e.flow))) > 0) {
            e.flow += f, edges[G[x][i] ^ 1].flow -= f;
            flow += f, a -= f;
            if (a == 0) break;
        }
    }
    return flow;
}

// 求s-t最大流。如果最大流超过 limit，则只找一个流量为 limit 的流
int Maxflow(int s, int t, int limit) {
    this->s = s, this->t = t;
    int flow = 0;
    while (BFS()) {
        memset(cur, 0, sizeof(cur));
        flow += DFS(s, limit - flow);
        if (flow == limit) break; // 达到流量限制，直接退出
    }
    return flow;
}

};

Dinic g;

const int maxm = 200 + 10;
int main() {
    int u[maxm], v[maxm]; // 输入边
    for (int n, m, k, S, T; scanf("%d%d%d%d%d", &n, &m, &k, &S, &T) == 5;) {
        for (int i = 0; i < m; i++) scanf("%d%d", &u[i], &v[i]);
        g.init();
    }
}

```



```

int day = 1;
g.clearNodes(0, n - 1); // 第一层点为 0~n-1。day 层 (day ≥ 1): day*n~day*n+n-1
int flow = 0;
for (;;) { // 判断 day 天是否有解
    // 一艘飞船最多需要 n-1 天到达目的星球, 沿着这一路线最多需要 n+k-2 天
    // 就可以运完所有飞船, 总结点数不超过 (n+k-1)n
    g.clearNodes(day * n, day * n + n - 1);
    for (int i = 0; i < n; i++)
        g.AddEdge((day - 1) * n + i, day * n + i, INF); // 原地不动
    for (int i = 0; i < m; i++) {
        g.AddEdge((day - 1) * n + u[i] - 1, day * n + v[i] - 1, 1); // u[i]→v[i]
        g.AddEdge((day - 1) * n + v[i] - 1, day * n + u[i] - 1, 1); // v[i]→u[i]
    }
    flow += g.Maxflow(S - 1, day * n + T - 1, k - flow);
    if (flow == k) break;
    day++;
}

// 输出解
printf("%d\n", day);
int idx = 0;
vector<int> location(k, S); // 每艘飞船的当前位置
for (int d = 1; d <= day; d++) {
    idx += n * 2;
    vector<int> moved(k, 0); // 第 d 天有没有移动飞船 i
    vector<int> a, b; // 第 d 天有一艘飞船从 a[i] 到 b[i]
    for (int i = 0; i < m; i++) {
        int f1 = g.edges[idx].flow;
        idx += 2;
        int f2 = g.edges[idx].flow;
        idx += 2;
        if (f1 == 1 && f2 == 0) a.push_back(u[i]), b.push_back(v[i]);
        if (f1 == 0 && f2 == 1) a.push_back(v[i]), b.push_back(u[i]);
    }
    printf("%d", a.size());
    for (int i = 0; i < a.size(); i++) {
        // 查找是哪艘飞船从 a[i] 移动到了 b[i]
        for (int j = 0; j < k; j++)
            if (!moved[j] && location[j] == a[i]) {
                printf(" %d %d", j + 1, b[i]);
                moved[j] = 1, location[j] = b[i];
                break;
            }
    }
    printf("\n");
}
return 0;
}

```

/*

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.6.4 节例题 31

同类题目：The K-League, POJ 1336

*/

例7-32 【拆点；最小费用流】生产销售规划 (Acme Corporation, UVa 11613)

Acme公司生产一种X元素。给出该元素在未来 M 个月中每个月单位售价、最大产量、生产成本、最大销售量以及最大储存时间（过期报废，不过可以储存任意多的量）。你的任务是计算出公司能够赚到的最大利润。

输入的第一行为数据组数 T ($T \leq 100$)。每组数据的第一行为两个整数 M 和 I ($M \leq 100$, $0 \leq I \leq 10^6$)，表示要考虑的月数以及每个单位的X元素存放一个月的代价。接下来的 M 行描述了每个月的参数，其中第 i 行包含5个整数 m_i, n_i, p_i, s_i, E_i ($0 \leq m_i, n_i, p_i, s_i \leq 10^6$, $0 \leq E_i \leq M$)， m_i 是第 i 月的单位生产成本， n_i 是最大产量， p_i 是销售单价， s_i 是当月最大销售量， E_i 是这个月生产的X元素能够储存的最大时间。例如，对于第1个月， $E_1=3$ ，这个月生产的X元素就只能在第1~4个月销售，到了第5个月就不能卖了。

对于每组数据，输出前 M 个月能够取得的最大利润。注意，只能考虑前 M 个月的销售。如果有任何元素的储存超过这个期限，无论卖不卖都得忽略。

【样例输入】

```
1
2 2
2 10 3 20 2
10 100 7 5 2
```

【样例输出】

Case 1: 20

【代码实现】

```
// 刘汝佳
#include <algorithm>
#include <cassert>
#include <cstdio>
#include <cstring>
#include <queue>
#include <vector>
using namespace std;
const int maxn = 202 + 10, INF = 1e9;
typedef long long LL;
struct Edge {
    int from, to, cap, flow, cost;
};

struct MCMF {
    int n, m, s, t;
    vector<Edge> edges;
```

```

vector<int> G[maxn];
int inq[maxn]; // 是否在队列中
int d[maxn]; // Bellman-Ford 算法
int p[maxn]; // 上一条弧
int a[maxn]; // 可改进量

void init(int n) {
    this->n = n;
    for (int i = 0; i < n; i++) G[i].clear();
    edges.clear();
}

void AddEdge(int from, int to, int cap, int cost) {
    edges.push_back((Edge) {from, to, cap, 0, cost});
    edges.push_back((Edge) {to, from, 0, 0, -cost});
    m = edges.size();
    G[from].push_back(m - 2);
    G[to].push_back(m - 1);
}

bool BellmanFord(int s, int t, LL& ans) {
    fill_n(inq, n + 1, false), fill_n(d, n + 1, INF);
    queue<int> Q;
    d[s] = 0, inq[s] = 1, p[s] = 0, a[s] = INF, Q.push(s);
    while (!Q.empty()) {
        int u = Q.front();
        Q.pop();
        inq[u] = 0;
        for (size_t i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]];
            if (e.cap > e.flow && d[e.to] > d[u] + e.cost) {
                d[e.to] = d[u] + e.cost, p[e.to] = G[u][i];
                a[e.to] = min(a[u], e.cap - e.flow);
                if (!inq[e.to]) Q.push(e.to), inq[e.to] = 1;
            }
        }
    }
    if (d[t] > 0) return false;
    ans += (LL)d[t] * (LL)a[t];
    int u = t;
    while (u != s) {
        edges[p[u]].flow += a[t], edges[p[u] ^ 1].flow -= a[t];
        u = edges[p[u]].from;
    }
    return true;
}

// 需要保证初始网络中没有负权圈
LL Mincost(int s, int t) {

```

```

    LL cost = 0;
    while (BellmanFord(s, t, cost))
        ;
    return cost;
}
};

MCMF g;

int main() {
    int T;
    scanf("%d", &T);
    for (int kase = 1, M, store_cost; kase <= T; kase++) {
        scanf("%d%d", &M, &store_cost);
        g.init(2 * M + 2);
        int source = 0, sink = 2 * M + 1;
        for (int i=1, make_cost, make_limit, price, sell_limit, max_store; i<=M; i++){
            scanf("%d%d%d%d%d", &make_cost, &make_limit, &price, &sell_limit, &max_store);
            g.AddEdge(source, i, make_limit, make_cost);
            g.AddEdge(M + i, sink, sell_limit, -price); // 收益是负费用
            for (int j = 0; j <= max_store; j++)
                if (i + j <= M) // 存 j 个月以后卖
                    g.AddEdge(i, M + i + j, INF, store_cost * j);
        }
        printf("Case %d: %lld\n", kase, -g.Mincost(source, sink));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 5.6.4 节例题 34
同类题目：Rent a Car, UVa 12433
*/

```

例 7-33 【网络流建模；EdmondsKarp】矩阵解压（Matrix Decompressing, UVa 11082）

对于一个 R 行 C 列的正整数矩阵（ $1 \leq R, C \leq 20$ ），设 A_i 为前 i 行所有元素之和， B_i 为前 i 列所有元素之和。已知 R, C 和数组 A 和 B ，找一个满足条件的矩阵。矩阵中的元素必须是 $1 \sim 20$ 的正整数。输入保证有解。

【代码实现】

```
// 使用 EdmondsKarp 的慢速版本
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

const int NN = 50 + 5, INF = 1e9;
```

```

struct Edge {
    int from, to, cap, flow;
    Edge(int u, int v, int c, int f): from(u), to(v), cap(c), flow(f) {}
};

struct EdmondsKarp {
    int n, m;
    vector<Edge> edges;    // 边数的两倍
    vector<int> G[NN];     // 邻接表, G[i][j]表示结点 i 的第 j 条边在 e 数组中的序号
    int a[NN];            // 当起点到 i 的可改进量
    int p[NN];            // 最短路树上 p 的入弧编号

    void init(int n) {
        for (int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int cap) {
        edges.push_back(Edge(from, to, cap, 0));
        edges.push_back(Edge(to, from, 0, 0));
        m = edges.size();
        G[from].push_back(m - 2), G[to].push_back(m - 1);
    }

    int Maxflow(int s, int t) {
        int flow = 0;
        for (;;) {
            memset(a, 0, sizeof(a));
            queue<int> Q;
            Q.push(s);
            a[s] = INF;
            while (!Q.empty()) {
                int x = Q.front(); Q.pop();
                for (size_t i = 0; i < G[x].size(); i++) {
                    Edge& e = edges[G[x][i]];
                    if (!a[e.to] && e.cap > e.flow) {
                        p[e.to] = G[x][i];
                        a[e.to] = min(a[x], e.cap - e.flow);
                        Q.push(e.to);
                    }
                }
            }
            if (a[t]) break;
        }
        if (!a[t]) break;
        for (int u = t; u != s; u = edges[p[u]].from) {
            edges[p[u]].flow += a[t];
            edges[p[u] ^ 1].flow -= a[t];
        }
        flow += a[t];
    }
}

```

```

        return flow;
    }
};

EdmondsKarp g;
int no[NN][NN];

int main() {
    int T; scanf("%d", &T);
    for (int kase = 1, R, C, v; kase <= T; kase++) {
        scanf("%d%d", &R, &C);
        g.init(R + C + 2);
        for (int i = 1, last = 0; i <= R; i++) {
            scanf("%d", &v);
            g.AddEdge(0, i, v - last - C);    // 行的和为 v-last
            last = v;
        }
        for (int i = 1, last = 0; i <= C; i++) {
            scanf("%d", &v);
            g.AddEdge(R + i, R + C + 1, v - last - R); // 列和为 v-last
            last = v;
        }
        for (int i = 1; i <= R; i++)
            for (int j = 1; j <= C; j++)    // no[i][j]是 cell(i,j)对应的边的编号
                g.AddEdge(i, R + j, 19), no[i][j] = g.edges.size() - 2;
        g.Maxflow(0, R + C + 1);

        printf("Matrix %d\n", kase);
        for (int i = 1; i <= R; i++) {
            for (int j = 1; j <= C; j++)
                printf("%d ", g.edges[no[i][j]].flow + 1); // 每个空格之前减过一
            puts("");
        }
        puts("");
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题11-8
同类题目: Chips Challenge, ACM/ICPC WF 2011, UVa 1104
*/

```

例7-34 【结点容量；拆点法；最小费用流】海军上将 (Admiral, ACM/ICPC NWERC 2012, UVa 1658)

给出一个包含 v ($3 \leq v \leq 1000$) 个点、 e ($3 \leq e \leq 10\ 000$) 条边的有向加权图，求1到 v 的两条不相交（除起点和终点外没有公共点）的路径，使得权值和最小。如图7-7所示，从1到6的两条最优路径为1-3-6（权和为33）和1-2-5-4-6（权和为53）。

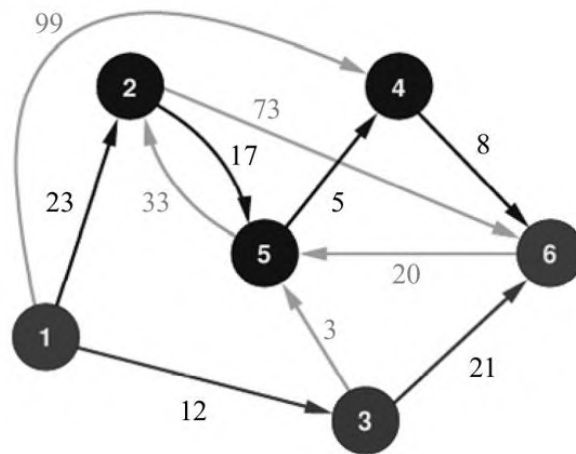


图7-7 从1到6的两条最优路径

【代码实现】

```

// 刘汝佳
using namespace std;
#include <bits/stdc++.h>
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
struct Edge {
    int from, to, cap, flow, cost;
    Edge(int u, int v, int c, int f, int co)
        : from(u), to(v), cap(c), flow(f), cost(co) {}
};

template <int maxn, int INF>
struct MCMF {
    int n, m;
    vector<Edge> edges;
    vector<int> G[maxn];
    bool inq[maxn];    // 是否在队列中
    int d[maxn];       // Bellman-Ford 算法
    int p[maxn];       // 上一条弧
    int a[maxn];       // 可改进量

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int cap, int cost) {
        edges.push_back(Edge(from, to, cap, 0, cost));
        edges.push_back(Edge(to, from, 0, 0, -cost));
        m = edges.size();
        G[from].push_back(m - 2), G[to].push_back(m - 1);
    }
};

```

```

bool BellmanFord(int s, int t, int& flow, LL& ans) {
    fill_n(d, n, INF), fill_n(inq, n, false);
    d[s] = 0, inq[s] = true, p[s] = 0, a[s] = INF;

    queue<int> Q;
    Q.push(s);
    while (!Q.empty()) {
        int u = Q.front();
        Q.pop();
        inq[u] = false;
        for (size_t i = 0; i < G[u].size(); i++) {
            Edge& e = edges[G[u][i]];
            if (e.cap > e.flow && d[e.to] > d[u] + e.cost) {
                d[e.to] = d[u] + e.cost, p[e.to] = G[u][i];
                a[e.to] = min(a[u], e.cap - e.flow);
                if (!inq[e.to]) Q.push(e.to), inq[e.to] = true;
            }
        }
    }
    if (d[t] == INF) return false; // s-t 不连通, 失败退出
    flow += a[t];
    ans += (LL)d[t] * (LL)a[t];
    int u = t;
    while (u != s) {
        edges[p[u]].flow += a[t], edges[p[u] ^ 1].flow -= a[t];
        u = edges[p[u]].from;
    }
    return true;
}

// 需要保证初始网络中没有负权圈
LL Mincost(int s, int t, int& flow) {
    LL ans = 0;
    flow = 0;
    while (BellmanFord(s, t, flow, ans))
        ;
    return ans;
}

};

const int MAXV = 1000 + 4, INF = 0x7f7f7f7f;
MCMF<2 * MAXV + 2, INF> solver;

int main() {
    for (int a, b, c, V, E, flow; cin >> V >> E;) {
        solver.init(2 * V + 2);
        int S = 1, T = V + V;
        solver.AddEdge(1, 1 + V, 2, 0);
        _for(v, 2, V) solver.AddEdge(v, v + V, 1, 0);
        solver.AddEdge(V, V + V, 2, 0);
    }
}

```

```

    _for(i, 0, E) cin >> a >> b >> c, solver.AddEdge(a + V, b, 1, c);
    LL ans = solver.Mincost(S, T, flow);
    assert(flow == 2);
    cout << ans << endl;
}
return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题11-9
同类题目: Transportation, HDU 3667
*/

```

例7-35 【最小割建模】水塘 (Pool construction, NWERC 2011, UVa 1515)

输入一个 h 行 w 列的字符矩阵，草地用“#”表示，洞用“.”表示，如图7-8所示。你可以把草地改成洞，每格花费为 d ；也可以把洞填埋成草地，每格花费为 f ；最后还需要在草地和洞之间修围栏，每个单位围栏的花费为 b 。整个矩阵第一行／列和最后一行／列必须都是草地。求最小花费。其中， $2 \leq w, h \leq 50$ ， $1 \leq d, f, b \leq 10\,000$ 。

```

# . . ##
## . ##
# . # . #
#####

```

图7-8 水塘问题示意图

例如， $d=1$ ， $f=8$ ， $b=1$ ，则图7-8中的最小花费为27，方法是先把第一行的洞填埋成草地（花费16），然后把第3行、第3列的草地挖成洞（花费1），再修10个单位的围栏（花费10）。

【代码实现】

```

// 刘汝佳
// 因为图较大，所以采用 Dinic 算法而不是 EdmondsKarp 算法
// 得益于接口一致性，读者无须理解 Dinic 就能使用它
#include <bits/stdc++.h>
using namespace std;

const int maxn = 50 * 50 + 10, INF = 1e9;
struct Edge {
    int from, to, cap, flow;
};
bool operator<(const Edge& a, const Edge& b) {
    return a.from < b.from || (a.from == b.from && a.to < b.to);
}

struct Dinic {
    int n, m, s, t;
    vector<Edge> edges;           // 边数的两倍
    vector<int> G[maxn];         // 邻接表，G[i][j]表示结点 i 的第 j 条边在 e 数组中的序号
    bool vis[maxn];              // BFS 使用
    int d[maxn];                 // 从起点到 i 的距离
    int cur[maxn];               // 当前弧指针

```

```

void init(int n) {
    for (int i = 0; i < n; i++) G[i].clear();
    edges.clear();
}

void AddEdge(int from, int to, int cap) {
    edges.push_back((Edge){from, to, cap, 0});
    edges.push_back((Edge){to, from, 0, 0});
    m = edges.size(), G[from].push_back(m - 2), G[to].push_back(m - 1);
}

bool BFS() {
    memset(vis, 0, sizeof(vis));
    queue<int> Q;
    Q.push(s), vis[s] = 1, d[s] = 0;
    while (!Q.empty()) {
        int x = Q.front();
        Q.pop();
        for (size_t i = 0; i < G[x].size(); i++) {
            Edge& e = edges[G[x][i]];
            if (!vis[e.to] && e.cap > e.flow)
                vis[e.to] = 1, d[e.to] = d[x] + 1, Q.push(e.to);
        }
    }
    return vis[t];
}

int DFS(int x, int a) {
    if (x == t || a == 0) return a;
    int flow = 0, f;
    for (int& i = cur[x]; (size_t)i < G[x].size(); i++) {
        Edge& e = edges[G[x][i]];
        if (d[x] + 1 == d[e.to] && (f = DFS(e.to, min(a, e.cap - e.flow))) > 0) {
            e.flow += f, edges[G[x][i] ^ 1].flow -= f;
            flow += f, a -= f;
            if (a == 0) break;
        }
    }
    return flow;
}

int Maxflow(int s, int t) {
    this->s = s, this->t = t;
    int flow = 0;
    while (BFS()) memset(cur, 0, sizeof(cur)), flow += DFS(s, INF);
    return flow;
}
};

```

```

Dinic g;
int w, h;
char pool[99][99];
inline int ID(int i, int j) { return i * w + j; }
int main() {
    int T, d, f, b;
    scanf("%d", &T);
    while (T--) {
        scanf("%d%d%d%d%d", &w, &h, &d, &f, &b);
        for (int i = 0; i < h; i++) scanf("%s", pool[i]);
        int cost = 0;
        for (int i = 0; i < h; i++) {
            if (pool[i][0] == '.') pool[i][0] = '#', cost += f;
            if (pool[i][w - 1] == '.') pool[i][w - 1] = '#', cost += f;
        }
        for (int i = 0; i < w; i++) {
            if (pool[0][i] == '.') pool[0][i] = '#', cost += f;
            if (pool[h - 1][i] == '.') pool[h - 1][i] = '#', cost += f;
        }
        g.init(h * w + 2);

        for (int i = 0; i < h; i++)
            for (int j = 0; j < w; j++) {
                if (pool[i][j] == '#') { // 草地 grass
                    int cap = INF;
                    if (i != 0 && i != h - 1 && j != 0 && j != w - 1) cap = d;
                    g.AddEdge(h * w, ID(i, j), cap); // s->grass, cap=d or inf
                } else { // 洞 hole
                    g.AddEdge(ID(i, j), h * w + 1, f); // hole->t, cap=f
                }
                if (i > 0) g.AddEdge(ID(i, j), ID(i - 1, j), b);
                if (i < h - 1) g.AddEdge(ID(i, j), ID(i + 1, j), b);
                if (j > 0) g.AddEdge(ID(i, j), ID(i, j - 1), b);
                if (j < w - 1) g.AddEdge(ID(i, j), ID(i, j + 1), b);
            }
        printf("%d\n", cost + g.Maxflow(h * w, h * w + 1));
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题11-12
同类题目: Cable TV Network, ACM/ICPC SEERC 2004, UVa 1660
*/

```

例7-36 【最小费用循环流】帮助小罗拉 (Help Little Laura, 北京2007, UVa 1659)

平面上有 m 条有向线段连接了 n 个点。你从某个点出发顺着有向线段行走，给沿途经过的每条线段涂一种不同的颜色，最后回到起点。你可以多次行走，给多个回路涂色。可以重复经过一个点，但不能重复经过一条有向线段。如图7-9所示，是一种涂色方法（虚线表示未涂色）。

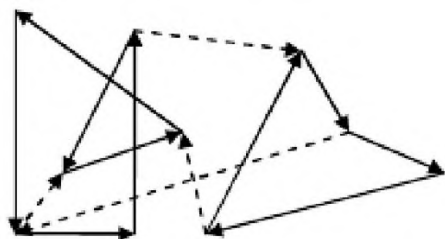


图7-9 涂色方法示意图

每涂一个单位长度，将得到 x 分。每使用一种颜料，将扣掉 y 分。假定颜料有无限多种，如何涂色才能使得分最大？输入保证若存在有向线段 $u \rightarrow v$ ，则不会出现有向线段 $v \rightarrow u$ 。 $n \leq 100$ ， $m \leq 500$ ， $1 \leq x, y \leq 1000$ 。

【代码实现】


```

// 算法一：改造网络，去掉负权
// 刘汝佳
#include <bits/stdc++.h>
using namespace std;

const int NN = 100 + 10, INF = 1e9;

struct Edge {
    int from, to, cap, flow;
    double cost;
    Edge(int u, int v, int c, int f, double w)
        : from(u), to(v), cap(c), flow(f), cost(w) {}
};

struct MCMF {
    int n, m;
    vector<Edge> edges;
    vector<int> G[NN];
    int inq[NN];    // 是否在队列中
    double d[NN];   // Bellman-Ford 算法
    int p[NN];      // 上一条弧
    int a[NN];      // 可改进量

    void init(int n) {
        this->n = n;
        for (int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int cap, double cost) {
        edges.push_back(Edge(from, to, cap, 0, cost));
        edges.push_back(Edge(to, from, 0, 0, -cost));
        m = edges.size();
        G[from].push_back(m - 2), G[to].push_back(m - 1);
    }

    bool BellmanFord(int s, int t, int& flow, double& cost) {
        for (int i = 0; i < n; i++) d[i] = INF;
        fill_n(inq, n + 1, 0);
        d[s] = 0, inq[s] = 1, p[s] = 0, a[s] = INF;
    }
};

```

```

queue<int> Q;
Q.push(s);
while (!Q.empty()) {
    int u = Q.front();
    Q.pop();
    inq[u] = 0;
    for (size_t i = 0; i < G[u].size(); i++) {
        Edge& e = edges[G[u][i]];
        if (e.cap > e.flow && d[e.to] > d[u] + e.cost) {
            d[e.to] = d[u] + e.cost, p[e.to] = G[u][i];
            a[e.to] = min(a[u], e.cap - e.flow);
            if (!inq[e.to]) Q.push(e.to), inq[e.to] = 1;
        }
    }
}
if (d[t] == INF) return false;
flow += a[t], cost += d[t] * a[t];
for (int u = t; u != s; u = edges[p[u]].from)
    edges[p[u]].flow += a[t], edges[p[u] ^ 1].flow -= a[t];
return true;
}

// 需要保证初始网络中没有负权圈
int MincostMaxflow(int s, int t, double& cost) {
    int flow = 0;
    cost = 0;
    while (BellmanFord(s, t, flow, cost))
        ;
    return flow;
}

};

MCMF g;
int x[NN], y[NN], c1[NN], c2[NN];
vector<int> G[NN];

int main() {
    for (int n, a, b, v, kase = 1; scanf("%d%d%d", &n, &a, &b) == 3; kase++) {
        g.init(n + 2);
        for (int u = 0; u < n; u++) {
            G[u].clear(), scanf("%d%d", &x[u], &y[u]);
            while (scanf("%d", &v) && v) G[u].push_back(v - 1);
        }
        fill_n(c1, n + 1, 0), fill_n(c2, n + 1, 0);
        double sum = 0;
        for (int u = 0; u < n; u++)
            for (size_t i = 0; i < G[u].size(); i++) {
                int v = G[u][i];

```

```

double d = sqrt((x[u]-x[v]) * (x[u]-x[v]) + (y[u]-y[v]) * (y[u]-y[v]));
double edge_cost = -d * a + b; // 最小化 sum{edge_cost}
if (edge_cost >= 0)
    g.AddEdge(u, v, 1, edge_cost);
else
    g.AddEdge(v, u, 1, -edge_cost), c1[v]++, c2[u]++, sum += -edge_cost;
}
for (int u = 0; u < n; u++) {
    if (c1[u] > c2[u]) g.AddEdge(n, u, c1[u] - c2[u], 0);
    if (c2[u] > c1[u]) g.AddEdge(u, n + 1, c2[u] - c1[u], 0);
}

double cost;
g.MincostMaxflow(n, n + 1, cost);
double ans = sum - cost;
if (ans < 0) ans = 0; // 避免出现 -0.0
printf("Case %d: %.2lf\n", kase, ans);
}
return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题 11-15
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表7-5所示。

表7-5 网络流问题例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 7-30	UVa 11248	Frequency Hopping	最大流：ISAP	刘汝佳
例 7-31	POJ 1895	Bring Them There	最大流：Dinic；图论建模	刘汝佳
例 7-32	UVa 11613	Acme Corporation	拆点：最小费用流	刘汝佳
例 7-33	UVa 11082	Matrix Decompressing	网络流建模；EdmondsKarp	刘汝佳
例 7-34	UVa 1658	Admiral	结点容量；拆点法；最小费用流	刘汝佳
例 7-35	UVa 1515	Pool construction	最小割建模	刘汝佳
例 7-36	UVa 1659	Help Little Laura	最小费用循环流	刘汝佳